Database Migrations with Flyway/Liquibase — PostgreSQL + pgAdmin

This continues your **JDBC + DAO + Transactions** learning.

The main idea is simple:

Instead of manually changing the database in pgAdmin, store every database change as a migration file inside your Java project.

That makes database changes reproducible for you, your team, CI/CD, testing, and production.

1. What Is a Database Migration?

Suppose today you create:

books

members

loans

Tomorrow you need:

books.available_copies

A beginner might open pgAdmin and manually execute:

```
ALTER TABLE books
ADD COLUMN available_copies INT;
```

That works on your computer.

But imagine a team:

Developer A → changes database

Developer B → doesn't know about change

Developer C → production database doesn't have change

CI server → database is different

Now problems appear.

A **migration** solves this by recording database changes as files.

2. Without Migrations

Imagine:

Developer A

↓

Changes PostgreSQL manually

↓

Works on A's laptop

But:

Developer B

↓

Clones project

↓

Database is different

You now have:

Code version ≠ Database version

3. With Migrations

Instead:

```
Project
│
├── Java code
│
├── pom.xml
│
└── db/migration
      │
      ├── V1__init_library.sql
      ├── V2__add_available_copies.sql
      └── V3__add_member_phone.sql
```

Everyone gets the same migration history.

```
V1
↓
V2
↓
V3
↓
Current database
```

4. What Is Flyway?

Flyway is a database migration tool.

It tracks which migration files have already been executed.

For example:

V1__init_library.sql

V2__add_available_copies.sql

V3__add_member_phone.sql

Flyway executes them in order.

It also maintains a table in PostgreSQL that records migration history.

Conceptually:

Flyway

↓

Migration files

↓

PostgreSQL

↓

flyway_schema_history

5. Flyway Naming Convention

A basic Flyway versioned migration looks like:

V1__description.sql

Examples:

V1__init_library.sql

V2__add_available_copies.sql

V3__add_member_phone.sql

V4__create_categories.sql

Important:

V1

means version 1.

Then:

__

two underscores.

Then:

description

Then:

.sql

So:

V1__init_library.sql

6. Maven Dependency

For a Maven Java project, add Flyway.

```
org.flywaydb
flyway-core
11.13.0
```

For PostgreSQL, depending on the Flyway version/setup, you may also need the PostgreSQL database support module:

```
org.flywaydb
```

```
    flyway-database-postgresql
    11.13.0
```

Keep the Flyway dependency versions the same.

You already have the PostgreSQL JDBC driver:

```
    org.postgresql
    postgresql
    42.7.7
```

7. Project Structure

Create:

```
library-jdbc/
│
├── pom.xml
│
└── src/
    └── main/
        ├── java/
        │   ├── Book.java
        │   ├── DatabaseConnection.java
        │   ├── BookDao.java
        │   ├── LoanDao.java
        │   ├── LibraryService.java
```

```
│   └── Main.java
│
└── resources/
    └── db/
        └── migration/
            ├── V1__init_library.sql
            ├── V2__add_available_copies.sql
            └── V3__seed_sample_data.sql
```

The important location is:

src/main/resources/db/migration

8. V1 — Initial Library Schema

Create:

V1__init_library.sql

Put:

```
CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    isbn VARCHAR(50) NOT NULL UNIQUE,
    published_year INT,
    total_copies INT NOT NULL
);
```

```
CREATE TABLE members (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE loans (
    id SERIAL PRIMARY KEY,
    book_id INT NOT NULL,
    member_id INT NOT NULL,
    loan_date DATE NOT NULL,
    return_date DATE,

    CONSTRAINT fk_loan_book
        FOREIGN KEY (book_id)
        REFERENCES books(id),

    CONSTRAINT fk_loan_member
        FOREIGN KEY (member_id)
        REFERENCES members(id)
);
```

This is your initial database structure.

9. V2 — Add a Column

Now suppose your application needs to know how many copies are currently available.

Create:

V2__add_available_copies.sql

Put:

```
ALTER TABLE books
ADD COLUMN available_copies INT NOT NULL DEFAULT 0;
```

Now the schema changes:

V1:

```
books
├── id
├── title
├── isbn
├── published_year
└── total_copies
```

After V2:

```
books
├── id
├── title
├── isbn
```

├── published_year

├── total_copies

└── available_copies

10. V3 — Seed Sample Data

Create:

V3__seed_sample_data.sql

INSERT INTO books

(title, isbn, published_year, total_copies, available_copies)

VALUES

('Clean Code', 'ISBN001', 2008, 5, 5),

('Effective Java', 'ISBN002', 2018, 4, 4),

('Java: The Complete Reference', 'ISBN003', 2020, 6, 6),

('Refactoring', 'ISBN004', 2018, 3, 3),

('Head First Java', 'ISBN005', 2019, 5, 5);

INSERT INTO members

(name, email)

VALUES

('Ali Khan', 'ali@example.com'),

('Ahmed Raza', 'ahmed@example.com'),

('Usman Malik', 'usman@example.com');

Now your migration history is:

V1

↓

Create tables

V2

↓

Add available_copies

V3

↓

Insert sample data

11. How Flyway Knows What Already Ran

Flyway creates a table similar to:

flyway_schema_history

You can inspect it in pgAdmin.

Run:

```
SELECT *
FROM flyway_schema_history
ORDER BY installed_rank;
```

You'll see migration information such as:

installed_rank | version | description

---|---|---

1 | 1 | init library

2 | 2 | add available copies

3 | 3 | seed sample data

The exact columns shown depend on the Flyway version.

This table is how Flyway keeps track of migration history.

12. Running Flyway from Java

For a simple learning project, you can run Flyway programmatically.

Create:

DatabaseMigration.java

```
import org.flywaydb.core.Flyway;

public class DatabaseMigration {

    public static void migrate() {

        Flyway flyway =
                Flyway.configure()
                        .dataSource(
                                "jdbc:postgresql://localhost:5432/library",
                                "postgres",
                                "your_password"
```

```
                )
                .load();

        flyway.migrate();
    }
}
```

Replace:

your_password

with your PostgreSQL password.

13. Run Migration Before Application Starts

Your Main.java can be:

```
public class Main {

    public static void main(String[] args) {

        DatabaseMigration.migrate();

        System.out.println(
                "Database migration completed"
        );
    }
}
```

Now:

Java starts

↓

Flyway starts

↓

Checks migration history

↓

Runs missing migrations

↓

Application continues

14. First Run

Suppose your database is empty.

Flyway sees:

V1

V2

V3

and executes:

V1

↓

V2

↓

V3

Database becomes:

Current database

↓

V3

15. Second Run

You run the application again.

Flyway checks:

V1 → already executed

V2 → already executed

V3 → already executed

Therefore:

Nothing to migrate

It doesn’t recreate the tables.

16. Add V4 Later

Suppose the business says:

We need a phone number for members.

Don’t modify:

V1__init_library.sql

Don’t modify:

V2__add_available_copies.sql

Instead create:

V4__add_member_phone.sql

with:

```
ALTER TABLE members
ADD COLUMN phone VARCHAR(30);
```

Now:

V1

↓

V2

↓

V3

↓

V4

Flyway sees:

V1 ✓

V2 ✓

V3 ✓

V4 ✗

and executes only V4.

17. The Golden Rule

This is one of the most important things to remember:

Never edit an old migration after it has already been applied. Create a new migration.

Suppose:

V1__init_library.sql

has already run.

Don’t change it from:

```
CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200)
);
```

to:

```
CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200),
    author VARCHAR(200)
);
```

Instead:

V2__add_author.sql

```
ALTER TABLE books
ADD COLUMN author VARCHAR(200);
```

18. Why Editing Old Migrations Is Dangerous

Imagine:

Developer A

V1 → already executed

Developer B changes V1.

Now:

Developer A database

≠

Developer B database

Flyway expects migration history to remain consistent.

Changing old migration files can cause checksum validation problems and, more importantly, destroys the historical record of how the schema evolved.

19. Migration History

Think of migrations like Git commits.

Git:

Commit 1

↓

Commit 2

↓

Commit 3

Database:

V1

↓

V2

↓

V3

You don’t normally rewrite old database history.

Instead:

New requirement

↓

New migration

20. Migration + Git

Migration files should be committed to Git along with your application.

Your repository:

```
Git Repository
│
├── src/
│   └── main/
│       ├── java/
│       └── resources/
│           └── db/
│               └── migration/
│                   ├── V1__init_library.sql
│                   ├── V2__add_available_copies.sql
│                   └── V3__seed_sample_data.sql
```

```
│
└── pom.xml
```

Then another developer clones the project:

```
git clone ...
```

runs the application:

```
Java application
      ↓
Flyway
      ↓
V1
V2
V3
      ↓
Database ready
```

21. Why This Is Important in CI/CD

Imagine your team deploys:

```
Application version 5
```

But the production database still has:

```
Database version 3
```

Application version 5 expects:

```
available_copies
```

but production doesn’t have it.

Application fails.

With migrations:

```
Deploy application
      ↓
Run migrations
      ↓
V4
      ↓
V5
      ↓
Start application
```

Now application and database schema can evolve together.

22. CI/CD Example

Imagine GitHub Actions:

```
Developer
  ↓
git push
  ↓
GitHub Actions
  ↓
```

Build

↓

Test

↓

Run database migrations

↓

Deploy application

The migration files are part of the application's source code.

This is why migrations are extremely useful in professional development.

23. Liquibase

Liquibase is another database migration/versioning tool.

The idea is similar:

Database changes

↓

Versioned changesets

↓

Liquibase

↓

PostgreSQL

But Liquibase commonly uses **changelogs and changesets**.

24. Flyway vs Liquibase

Flyway

SQL-first approach:

V1__init_library.sql

V2__add_available_copies.sql

V3__add_phone.sql

You write normal SQL.

Example:

```
ALTER TABLE members
ADD COLUMN phone VARCHAR(30);
```

Liquibase

Changelog-based approach.

For example, XML:

```
        name="phone"
        type="varchar(30)"
    />
```

Liquibase also supports formats such as YAML, JSON, XML, and formatted SQL.

25. Liquibase Changeset

A **changeset** represents a specific database change.

Conceptually:

```
changelog
   │
   ├── changeset 1
   ├── changeset 2
   └── changeset 3
```

Example:

```
    id="1"
    author="muneeb">




            name="id"
            type="int">


               primaryKey="true"
               nullable="false"/>
```

```
        name="title"
        type="varchar(200)"/>
```

You don’t need to become an expert in Liquibase now.

For your current JDBC learning, understand the concept.

26. Flyway vs Liquibase — Interview Comparison

Flyway	Liquibase
Simple	More feature-rich
SQL-first	Changelog/changeset based
Easy to learn	More configuration/options
Versioned SQL files	Changesets
Excellent for straightforward migrations	Useful for complex database change management

A good beginner interview answer:

Flyway is generally SQL-first and uses versioned migration files such as `V1__init.sql`, while Liquibase uses changelogs containing changesets. Both help version, track, and automate database schema changes.

27. What Is Rollback?

Rollback means:

Reverse a database change.

Suppose:

```
ALTER TABLE members

ADD COLUMN phone VARCHAR(30);
```

Rollback conceptually means:

```
ALTER TABLE members

DROP COLUMN phone;
```

But there is an important distinction.

You should **not think of migrations as simply editing old files to undo history**.

Instead, depending on the migration tool and workflow, you can explicitly define or execute rollback behavior.

For beginner-level understanding:

```
Migration
    ↓
Schema changes


Rollback
    ↓
Undo schema change
```

28. Important Migration Rule

Don’t do this:

V1

↓

V2

↓

V3

Then discover V2 was wrong and edit:

V2

Instead:

V1

↓

V2

↓

V3

↓

V4 → correct the problem

This preserves the history.

29. Flyway and Transactions

Database migrations often execute SQL changes in controlled migration operations.

For PostgreSQL, many schema operations are transactional, but not every database operation behaves identically across databases.

For your beginner level, remember:

The migration tool manages migration execution and records migration state; don’t manually mix application business transactions with schema migrations.

30. CRUD After Migration

Once Flyway has created:

books

members

loans

your existing JDBC code can continue working.

For example:

```
JdbcBookRepository repository =
        new JdbcBookRepository();

List books =
        repository.findAll();

books.forEach(
        System.out::println
);
```

The flow becomes:

Application starts

↓

Flyway migration

↓

PostgreSQL schema ready

↓

Repository

↓

JDBC

↓

CRUD

31. Updating Book Java Class

Because V2 added:

available_copies

your Java class should eventually contain:

private int availableCopies;

For example:

```
public class Book {

    private int id;
    private String title;
    private String isbn;
```

```
    private int publishedYear;
    private int totalCopies;
    private int availableCopies;

    public Book() {
    }

    public Book(
            int id,
            String title,
            String isbn,
            int publishedYear,
            int totalCopies,
            int availableCopies
    ) {
        this.id = id;
        this.title = title;
        this.isbn = isbn;
        this.publishedYear = publishedYear;
        this.totalCopies = totalCopies;
        this.availableCopies = availableCopies;
    }
```

```
public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getIsbn() {
    return isbn;
}

public void setIsbn(String isbn) {
    this.isbn = isbn;
}
```

```
public int getPublishedYear() {
    return publishedYear;
}

public void setPublishedYear(int publishedYear) {
    this.publishedYear = publishedYear;
}

public int getTotalCopies() {
    return totalCopies;
}

public void setTotalCopies(int totalCopies) {
    this.totalCopies = totalCopies;
}

public int getAvailableCopies() {
    return availableCopies;
}

public void setAvailableCopies(int availableCopies) {
    this.availableCopies = availableCopies;
```

```
    }

    @Override
    public String toString() {
        return "Book{" +
                "id=" + id +
                ", title='" + title + '\'' +
                ", isbn='" + isbn + '\'' +
                ", publishedYear=" + publishedYear +
                ", totalCopies=" + totalCopies +
                ", availableCopies=" + availableCopies +
                '}';
    }
}
```

32. Update Repository Query

Your previous query:

```
SELECT id, title, isbn, published_year, total_copies
FROM books
```

now becomes:

```
SELECT
    id,
```

```
    title,
    isbn,
    published_year,
    total_copies,
    available_copies
FROM books
```

And mapping:

```
private Book mapRow(ResultSet resultSet)
        throws SQLException {

    return new Book(
            resultSet.getInt("id"),
            resultSet.getString("title"),
            resultSet.getString("isbn"),
            resultSet.getInt("published_year"),
            resultSet.getInt("total_copies"),
            resultSet.getInt("available_copies")
    );
}
```

This demonstrates an important real-world point:

Database schema change

↓

Migration

↓

Java model may need change

↓

Repository may need change

33. Complete Migration Flow

This is the most important flow to understand:

Developer creates V1

↓

Commit to Git

↓

Application starts

↓

Flyway checks database

↓

V1 executed

↓

Developer creates V2

↓

Commit to Git

↓

Application starts

↓

Flyway sees V2 is new

↓

V2 executed

Eventually:

Git Repository

↓

Migration Files

↓

Flyway

↓

PostgreSQL

↓

Current Schema

34. Manual pgAdmin vs Migration

Manual approach

Open pgAdmin

↓

Write SQL

↓

Execute

Problem:

Who executed it?

When?

Which version?

Did everyone execute it?

Did production execute it?

Migration approach

V4__add_phone.sql

↓

Git

↓

Flyway

↓

PostgreSQL

Everything is documented and reproducible.

35. Common Mistakes

❌ Editing old migrations

V1__init.sql

already executed → don't modify it.

❌ Naming incorrectly

Don’t randomly use:

initial.sql

database1.sql

update.sql

Use:

V1__init_library.sql

V2__add_available_copies.sql

❌ Not committing migrations

If migration files aren’t in Git:

Developer A

↓

Has schema change

Developer B

↓

Doesn't have schema change

❌ Manually changing production

Avoid:

Developer

↓

SSH production

↓

Run random SQL

Instead:

Migration

↓

Review

↓

Version control

↓

Deployment

36. Interview Questions

Q1. What is a database migration?

A database migration is a version-controlled change to a database schema or data that can be applied consistently across environments.

Q2. Why are migrations important?

They make database changes reproducible, trackable, and consistent across developer machines, testing, CI/CD, staging, and production environments.

Q3. What is Flyway?

Flyway is a database migration tool that tracks and applies versioned database migrations.

Q4. What is the Flyway naming convention?

Example:

V1__init_library.sql

Where:

V1

is the version and:

init_library

is the description.

Q5. What happens if V1 and V2 already ran?

If you create:

V3__add_phone.sql

Flyway recognizes V1 and V2 as already applied and executes V3.

Q6. Should you edit V1 after it has run?

No. Create a new migration instead.

For example:

V1

V2

V3

rather than modifying V1.

Q7. Why commit migration files to Git?

So the database schema changes are versioned together with the application code and can be reproduced by other developers and deployment environments.

Q8. What is Liquibase?

Liquibase is a database change management tool that uses changelogs and changesets to define and track database changes.

Q9. Flyway vs Liquibase?

Flyway commonly follows a SQL-first versioned migration approach, while Liquibase uses changelogs and changesets and provides more abstraction and database change-management features.

Q10. Why are migrations important in CI/CD?

CI/CD environments need a predictable database schema. Migrations allow database changes to be automatically applied during deployment so the application's expected schema exists in staging or production.

37. Strong Interview Scenario

Interviewer:

Your application works on your machine, but after deploying it to production, it crashes because a new database column doesn't exist. How would you prevent this?

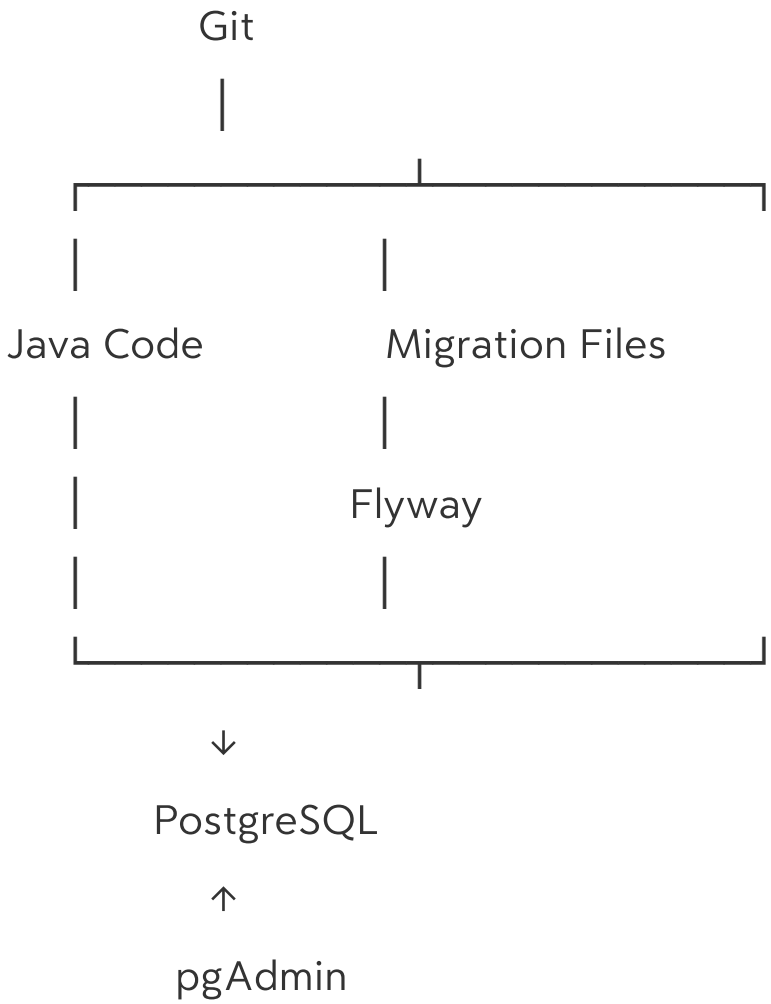
Strong answer:

I would use database migrations such as Flyway or Liquibase. Every schema change would be stored as a version-controlled migration and committed with the application code. During deployment, the migration tool would apply any pending migrations to the target database before the application starts using the new schema.

That’s a very good practical answer.

38. Persistence Checkpoint

At this point your architecture has evolved considerably:



And your application architecture:

DAO / Repository

↓

JDBC

↓

PostgreSQL

Database setup:

Flyway

↓

V1

↓

V2

↓

V3

↓

Current Schema

39. Final Cheat Sheet

Concept	Meaning
Migration	Version-controlled database change
Flyway	Migration/versioning tool
V1	First migration
V1__init.sql	Flyway versioned SQL migration

V2	Second schema change
Changeset	Liquibase unit of database change
Changelog	Liquibase collection of changesets
Rollback	Reverse a database change
flyway_schema_history	Flyway migration history table
Git	Stores migration history with application
CI/CD	Automatically applies migrations during deployment

Most important rules

1. Never manually depend on pgAdmin changes for team projects.

2. Put schema changes in migration files.

3. Commit migration files to Git.

4. Use V1, V2, V3... for Flyway migrations.

5. Never edit an already-applied migration.

6. Create a new migration for every new schema change.

7. Flyway tracks which migrations have already run.

8. Liquibase uses changelogs and changesets.

9. Migrations make environments reproducible.

10. Migrations are extremely useful for CI/CD.

The interview sentence to remember:

Database migrations provide a version-controlled history of database changes, allowing teams and CI/CD environments to reliably reproduce the same database schema without manually applying SQL changes.